



Università
degli Studi di
Messina

Università degli Studi di Messina

Dipartimento MIFT - Corso di Laurea Triennale in Informatica (L-31)

Laboratorio di Intelligenza Artificiale

PROGETTO 1:

Segmentazione Clienti e Churn Prediction

Massimo Mantineo

Matricola: 541924

Messina, A.A. 2025/2026

Indice

1	Descrizione del problema	3
1.1	Il Task Non Supervisionato (Clustering)	3
1.2	Il Task Supervisionato (Classificazione)	3
1.3	Le Metriche di Valutazione della Classificazione	3
2	Dataset e preparazione dei dati	4
2.1	Struttura e Principali Variabili	4
2.2	Operazioni di Preprocessing e Risposte Metodologiche	4
2.2.1	Gestione dei valori mancanti ed Imputazione	5
2.2.2	Codifica delle variabili categoriche (One-Hot Encoding)	5
2.2.3	Standardizzazione (Standard Scaling)	6
2.3	Criticità: Sbilanciamento delle Classi	7
3	Metodologia	7
3.1	L'architettura della Pipeline	7
3.2	I Modelli Scelti	7
3.2.1	k-Means (Clustering)	7
3.2.2	Regressione Logistica (Baseline)	8
3.2.3	Decision Tree (Modello Principale 1)	8
3.2.4	Multi-Layer Perceptron (Modello Principale 2)	8
3.3	Riproducibilità, Dipendenze e Istruzioni d'Esecuzione	9
4	Esperimenti	9
4.1	Setup Sperimentale e Prevenzione dell'Overfitting	9
4.2	Caratterizzazione dei due Cluster ($k = 2$)	9
5	Risultati	10
5.1	Risultati del Clustering k-Means	10
5.2	Risultati della Classificazione (Train vs. Test)	11
5.3	Analisi Critica dell'Overfitting e dell'Impatto del Clustering	11
6	Discussione critica	13
6.1	Punti di Forza della Soluzione	13
6.2	Limiti Sperimentali e Fonti di Errore	13
6.3	Interpretazione e Confronto: Decision Tree vs. MLP	13
6.4	Miglioramenti Futuri	14
6.5	Uso di strumenti di Intelligenza Artificiale Generativa	14
7	Conclusioni	14

Sommario

Il presente elaborato descrive lo sviluppo e i risultati di un progetto accademico focalizzato sull'integrazione di techniques di apprendimento non supervisionato (clustering) e supervisionato (classificazione) per l'analisi del comportamento della clientela in un'azienda di telecomunicazioni. Utilizzando il dataset pubblico *Telco Customer Churn*, abbiamo dapprima segmentato i clienti tramite l'algoritmo *k*-Means, individuando due cluster principali caratterizzati da comportamenti e profili di spesa differenti. Successivamente, abbiamo sviluppato e confrontato tre modelli predittivi di classificazione per stimare la probabilità di abbandono (*churn*) dei clienti: una baseline lineare (Regressione Logistica), un Albero di Decisione (Decision Tree) e una rete neurale artificiale (Multi-Layer Perceptron - MLP). I modelli sono stati valutati e confrontati in due scenari sperimentali: con e senza l'aggiunta del cluster ottimale come feature predittiva. I risultati mostrano che mentre la baseline e l'albero di decisione mantengono prestazioni stabili ($\approx 80.6\%$ di accuratezza), l'MLP trae un vantaggio qualitativo significativo dall'integrazione delle feature di clustering, registrando un incremento marcato del richiamo (*Recall*) da 0.4278 a 0.5829 ($\approx +15.5\%$). L'elaborato si conclude con un'analisi critica circa l'interpretabilità dei modelli, i limiti sperimentali e le future direzioni di sviluppo.

1 Descrizione del problema

Nel moderno panorama industriale e dei servizi, in particolare nel settore delle telecomunicazioni (Telco), la fidelizzazione del cliente è un fattore critico per la sostenibilità finanziaria. Acquisire un nuovo cliente comporta costi di marketing e infrastruttura notevolmente superiori rispetto al mantenimento di un cliente attivo. Per questa ragione, lo sviluppo di sistemi in grado di anticipare l'abbandono da parte del cliente, fenomeno noto come *customer churn*, riveste un ruolo primario per la business intelligence aziendale.

Questo progetto affronta il problema da una doppia prospettiva, combinando l'apprendimento non supervisionato e l'apprendimento supervisionato in una pipeline integrata.

1.1 Il Task Non Supervisionato (Clustering)

L'obiettivo dell'apprendimento non supervisionato è identificare pattern intrinseci o segmenti naturali nella clientela, senza disporre preventivamente di un'etichetta di classe. Questa operazione risponde alla domanda: *“Esistono profili di clientela simili tra loro per comportamento, tipologia di contratto e consumi?”*.

La segmentazione dei clienti (*Customer Segmentation*) viene qui affrontata tramite l'algoritmo *k*-Means, testando soluzioni con 2, 3 e 4 cluster. La scelta del numero ottimale di cluster viene supportata quantitativamente da:

- **Inertia (Metodo del Gomito - Elbow Method)**: la somma dei quadrati delle distanze dei campioni dal centro del loro cluster più vicino. Rappresenta una misura di coerenza interna dei gruppi.
- **Silhouette Score**: una metrica che misura quanto un elemento sia simile al proprio cluster (coesione) rispetto a quanto sia dissimile dagli altri cluster (separazione). Varia tra -1 e +1; valori vicini a +1 indicano un clustering ben definito.

1.2 Il Task Supervisionato (Classificazione)

Il task supervisionato consiste nel risolvere un problema di classificazione binaria. L'obiettivo è addestrare un classificatore che, dato il profilo di un cliente (un vettore di feature che include dati anagrafici, servizi sottoscritti, parametri finanziari e, opzionalmente, il cluster di appartenenza), predica se il cliente abbandonerà l'operatore (classe 1, *Churn: Yes*) o rimarrà fedele (classe 0, *Churn: No*).

1.3 Le Metriche di Valutazione della Classificazione

Per valutare l'efficacia dei modelli di classificazione, ci avvaliamo delle metriche classiche definite nel programma del corso (in particolare nelle slide sulla classificazione):

- **Accuracy (Accuratezza)**: Frazione delle predizioni corrette sul totale dei casi testati. Fornisce una misura della performance globale del modello:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (1)$$

dove *TP*, *TN*, *FP* e *FN* sono rispettivamente i Veri Positivi, Veri Negativi, Falsi Positivi e Falsi Negativi ricavati dalla matrice di confusione.

- **Recall (Richiamo o Sensibilità):** Frazione dei clienti che abbandonano (reali positivi) correttamente individuati dal modello:

$$\text{Recall} = \frac{TP}{TP + FN} \quad (2)$$

Nel contesto del churn prediction, la Recall è la metrica di business più rilevante. Un Falso Negativo (un cliente che sta per abbandonare ma che il modello classifica come fedele) rappresenta un cliente perso senza che l'azienda possa avviare azioni di fidelizzazione.

- **AUC (Area Under the ROC Curve):** L'area sotto la curva caratteristica del ricevitore (ROC), che descrive il trade-off tra il tasso di veri positivi (Recall) e il tasso di falsi positivi a varie soglie di classificazione. Un valore di AUC pari a 1.0 indica un classificatore perfetto, mentre un valore pari a 0.5 equivale ad una classificazione casuale. L'AUC è una metrica robusta nei confronti dello sbilanciamento delle classi.

2 Dataset e preparazione dei dati

Il dataset utilizzato è il *Telco Customer Churn* (distribuito storicamente da IBM e reperibile su Kaggle). Il dataset originale contiene 7043 osservazioni (clienti) e 21 variabili complessive.

2.1 Struttura e Principali Variabili

Ciascun cliente è descritto da attributi che possono essere suddivisi in tre macro-categorie:

1. **Variabili numeriche continue:** Mesi di permanenza (*tenure*), costi mensili (*MonthlyCharges*) e costi totali (*TotalCharges*).
2. **Variabili categoriche qualitative:** Tipo di contratto (*Contract*: mensile, annuale, biennale), metodo di pagamento (*PaymentMethod*), tipo di connessione internet (*InternetService*) e servizi accessori (linee multiple, sicurezza online, backup, protezione dei dispositivi, supporto tecnico, streaming TV e film).
3. **Variabili categoriche binarie:** Genere (*gender*), presenza di partner (*Partner*), presenza di familiari a carico (*Dependents*), fatturazione elettronica (*PaperlessBilling*) e fascia d'età anziana (*SeniorCitizen*, codificata numericamente in {0, 1}).

La variabile target è **Churn**, che assume valore "Yes" se il cliente ha abbandonato la compagnia nell'ultimo mese, "No" in caso contrario.

2.2 Operazioni di Preprocessing e Risposte Metodologiche

La preparazione dei dati è uno step cruciale per garantire la convergenza degli algoritmi ed evitare errori di calcolo. Di seguito descriviamo in modo dettagliato le operazioni effettuate.

2.2.1 Gestione dei valori mancanti ed Imputazione

Durante l'ispezione iniziale, la variabile *TotalCharges* presentava 11 righe con celle contenenti spazi vuoti (" ").

- **Analisi logica di business:** Incrociando queste righe con la variabile *tenure*, si è constatato che per tutti questi clienti la permanenza era pari a 0 mesi (nuovi clienti appena contrattualizzati). Abbiamo pertanto imputato questi valori mancanti a 0.0 (poiché non hanno ancora ricevuto addebiti) e convertito la colonna in formato numerico a virgola mobile (*float*).
- **Che cos'è l'imputazione con mediana e come si calcola il valore mediano?** L'imputazione con la mediana è una tecnica standard in cui i valori mancanti di una variabile quantitativa vengono sostituibili con la mediana della distribuzione della variabile stessa (calcolata sui dati non mancanti).

Matematicamente, per calcolare il valore mediano: 1. Si ordinano gli N valori della variabile in ordine non decrescente: $x_1 \leq x_2 \leq \dots \leq x_N$. 2. Se N è dispari, la mediana è il valore centrale in posizione $\frac{N+1}{2}$:

$$\text{Mediana} = x_{\frac{N+1}{2}} \quad (3)$$

3. Se N è pari, la mediana è la media aritmetica dei due valori centrali in posizione $\frac{N}{2}$ e $\frac{N}{2} + 1$:

$$\text{Mediana} = \frac{x_{\frac{N}{2}} + x_{\frac{N}{2}+1}}{2} \quad (4)$$

- **Perché si usa la mediana invece della media?** La mediana è una misura di tendenza centrale estremamente **robusta nei confronti degli outlier** (valori anomali o estremi). A differenza della media aritmetica (che risente pesantemente di un singolo valore molto alto o basso), la mediana risente solo dell'ordinamento dei dati. Nel nostro caso specifico, tuttavia, abbiamo preferito l'imputazione costante a 0.0 perché supportata da una logica di business certa (*tenure* = 0 implica zero consumi), evitando di distorcere la realtà con la mediana generale del dataset ($\approx 1397.47\$$).

2.2.2 Codifica delle variabili categoriche (One-Hot Encoding)

Le variabili categoriche qualitative non possono essere elaborate direttamente da modelli matematici come la Regressione Logistica o MLP. Abbiamo applicato il **One-Hot Encoding**:

- **In cosa consiste?** Consiste nel mappare una variabile qualitativa con C categorie distinte in C variabili binarie (dette *dummy*) aventi valore 1 se l'osservazione appartiene alla categoria e 0 altrimenti.
- **Perché escludiamo una categoria (`drop_first=True`)?** Per evitare la trappola della multicollinearità (perfetta dipendenza lineare tra le feature). Se tenessimo tutte le C colonne dummy, la loro somma darebbe sempre 1, che è perfettamente allineata al termine di intercetta (bias) dei modelli lineari,

rendendo la matrice delle feature non invertibile. Escludendo la prima colonna, questa funge da categoria di riferimento (baseline).

- **Come verifichiamo la correttezza del One-Hot Encoding nel codice?**

Abbiamo utilizzato la funzione standard `pd.get_dummies()` di pandas. Per verificare che abbia operato correttamente:

1. Si controlla lo shape del dataset post-encoding: se avevamo una colonna con C classi, il numero totale di colonne deve aumentare esattamente di $C - 1$. Nel nostro caso, il dataset è passato da 20 feature originarie a 30 colonne finali preelaborate.
2. Si ispeziona il dataset (`X_encoded.head()`) verificando che le nuove colonne contengano solo valori appartenenti all'insieme $\{0, 1\}$ e che per ogni record non vi sia mai più di un singolo valore 1 attivo tra le dummy di una stessa feature originaria.

2.2.3 Standardizzazione (Standard Scaling)

I modelli basati sulle distanze (k -Means) e sulle reti neurali (MLP) risentono fortemente delle differenze di scala tra le variabili.

- **Come funziona lo Standard Scaling?**

La standardizzazione trasforma una variabile quantitativa continua in modo che abbia media $\mu = 0$ e deviazione standard $\sigma = 1$. Per ciascun valore x_i , si calcola il valore scalato z_i :

$$z_i = \frac{x_i - \mu}{\sigma} \quad (5)$$

dove:

$$\mu = \frac{1}{N} \sum_{i=1}^N x_i \quad (\text{Media}) \quad (6)$$

$$\sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2} \quad (\text{Deviazione Standard}) \quad (7)$$

- **Cos'è la deviazione standard?**

È una misura della dispersione dei dati attorno alla loro media aritmetica. Indica, in media, quanto i singoli valori si discostino dal valore centrale.

- **Perché lo splitting in Train/Test si fa PRIMA dello scaling (Standardization)?**

Questa è una regola aurea del Machine Learning per evitare il **Data Leakage** (trapelamento di informazioni). Se calcolassimo la media μ e la deviazione standard σ sull'intero dataset (prima dello split), le informazioni del Test Set (che dovrebbe essere rigorosamente invisibile e simulare dati futuri sconosciuti) influenzerebbero i valori scalati del Training Set.

Pertanto: 1. Si effettua prima lo split del dataset. 2. Si calcolano media e deviazione standard (operazione di `fit`) **esclusivamente sul Training Set**. 3. Si applica la trasformazione (operazione di `transform`) sia sul Training Set che sul Test Set utilizzando i parametri calcolati sul Training Set.

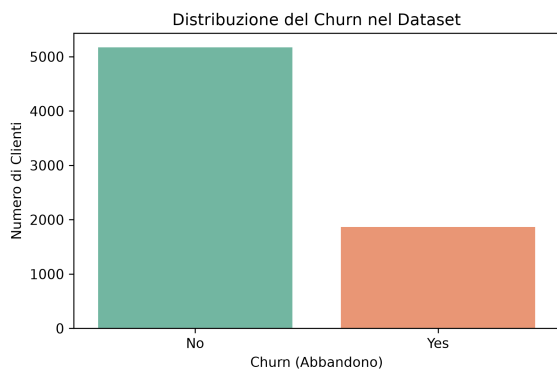


Figura 1: Distribuzione del Churn nel dataset.

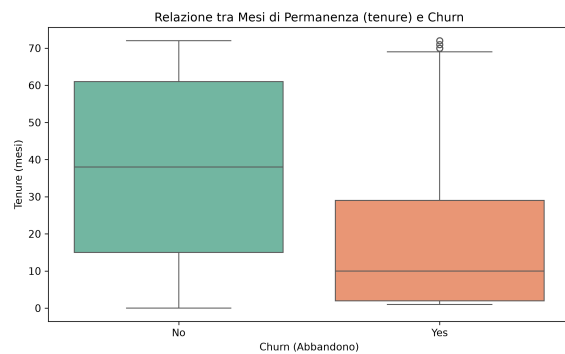


Figura 2: Relazione tra mesi di permanenza (tenure) e Churn.

2.3 Criticità: Sbilanciamento delle Classi

La variabile target *Churn* presenta una forte asimmetria nella sua distribuzione. Nel dataset, il 73.4% dei clienti appartiene alla classe 0 (*No Churn*) e solo il 26.6% appartiene alla classe 1 (*Churn*). Questa proporzione indica la presenza di uno sbilanciamento di circa 3:1. Per contrastare questa criticità, lo split train-test è stato eseguito in modo stratificato, garantendo la parità delle proporzioni delle classi nei due set.

3 Metodologia

La pipeline di analisi implementata prevede un flusso sequenziale suddiviso in tre macro-blocchi: pre-elaborazione, segmentazione non supervisionata e classificazione supervisionata nei due scenari.

3.1 L'architettura della Pipeline

L'architettura prevede il preprocessing delle feature (imputazione, codifica one-hot, scaling), l'applicazione di *k*-Means per generare la feature del cluster, l'aggiunta di questa feature dummy al dataset e l'addestramento dei classificatori.

3.2 I Modelli Scelti

3.2.1 k-Means (Clustering)

L'algoritmo *k*-Means è un metodo di partizionamento dello spazio delle feature in *k* gruppi disgiunti. L'algoritmo opera in modo iterativo partendo da *k* centroidi posizionati casualmente. Ad ogni iterazione:

1. Assegna ciascun punto al centroide più vicino (in base alla distanza euclidea).
2. Ricalcola la posizione di ciascun centroide come media geometrica dei punti ad esso assegnati.

Il processo termina quando la posizione dei centroidi si stabilizza. La scelta del *k* ottimale si basa sul Silhouette Score e sull'Inertia.

3.2.2 Regressione Logistica (Baseline)

Utilizzata come baseline semplice e interpretabile. Calcola una combinazione lineare delle feature di input e mappa il risultato nell'intervallo $[0, 1]$ tramite la funzione sigmoide:

$$\sigma(t) = \frac{1}{1 + e^{-t}} \quad (8)$$

Rappresenta la probabilità stimata che il cliente appartenga alla classe di abbandono.

3.2.3 Decision Tree (Modello Principale 1)

L'albero di decisione suddivide lo spazio delle feature in regioni rettangolari tramite regole logiche di split sequenziali (struttura a IF-THEN) che massimizzano la purezza dei nodi figli (Indice di Gini). Abbiamo impostato una profondità massima del modello pari a 5 (`max_depth=5`) per limitare l'overfitting.

3.2.4 Multi-Layer Perceptron (Modello Principale 2)

Il Multi-Layer Perceptron (MLP) è una rete neurale artificiale feedforward composta da layer sequenziali di nodi (neuroni).

- **Come viene addestrato?**

L'addestramento avviene in due fasi ripetute iterativamente: 1. **Forward Propagation:** I dati fluiscono dal layer di input attraverso i layer nascosti (hidden layers), dove ciascun neurone calcola la somma pesata degli input più il bias e applica una funzione di attivazione non lineare (nel nostro caso la funzione ReLU: $f(x) = \max(0, x)$). L'output finale viene calcolato nel layer di output. 2. **Backpropagation:** Si calcola l'errore tra la predizione e la classe reale tramite la funzione di perdita di cross-entropia. L'errore viene propagato a ritroso nella rete per calcolare i gradienti rispetto ai pesi e ai bias. I parametri vengono aggiornati tramite un ottimizzatore (Adam) basato sulla discesa del gradiente.

- **E se uno non usa la funzione di attivazione Sigmoide nel layer di output?**

Per problemi di classificazione binaria, la funzione di attivazione del layer di output deve essere necessariamente la **Sigmoide**. Essa comprime l'output reale della rete in un intervallo chiuso $[0, 1]$, interpretabile come probabilità a posteriori del target: $P(Y = 1|X)$. Se non usassimo la sigmoide, l'output della rete potrebbe assumere qualsiasi valore reale in $] - \infty, +\infty[$, impedendo una classificazione probabilistica diretta e l'uso della perdita di cross-entropia standard.

- **Perché servono le attivazioni non lineari nei layer nascosti?**

Senza funzioni di attivazione non lineari (es. ReLU, Tanh) nei layer nascosti, la composizione di più layer lineari collapserebbe matematicamente in una singola trasformazione lineare. La rete neurale perderebbe così la capacità di apprendere relazioni complesse e non lineari, diventando equivalente ad un semplice modello di Regressione Logistica.

3.3 Riproducibilità, Dipendenze e Istruzioni d'Esecuzione

In conformità ai requisiti di riproducibilità del codice, il notebook principale del progetto (`segmentazione_crunch.ipynb`) è strutturato per consentire l'esecuzione sequenziale di tutte le celle di codice. Il dataset viene scaricato automaticamente dai repository IBM all'avvio qualora non fosse già presente localmente nella cartella `dati/`. Le librerie fondamentali utilizzate con le rispettive versioni di riferimento sono:

- **Pandas** ($\geq 1.3.0$) e **NumPy** ($\geq 1.20.0$) per l'elaborazione e manipolazione tabellare.
- **Scikit-Learn** ($\geq 1.0.0$) per l'addestramento degli algoritmi di clustering (k -Means) e classificazione (Logistic Regression, Decision Tree, MLP Classifier).
- **Matplotlib** ($\geq 3.4.0$) e **Seaborn** ($\geq 0.11.0$) per la generazione di grafici e plot visivi.

Tutte le dipendenze con le versioni esatte utilizzate sono raccolte nel file `requirements.txt` e le istruzioni di installazione e avvio dell'ambiente virtuale sono dettagliate nel file `README.md` nella radice del progetto.

4 Esperimenti

4.1 Setup Sperimentale e Prevenzione dell'Overfitting

La verifica del comportamento dei modelli richiede un setup rigoroso basato sul monitoraggio dei fenomeni di **overfitting** e **underfitting**:

- **Che cos'è l'Overfitting?**
Si verifica quando un modello apprende eccessivamente i dettagli e il rumore del Training Set, al punto da perdere capacità di generalizzazione su dati nuovi (Test Set). Si manifesta con performance altissime nel training e significativamente inferiori nel test.
- **Che cos'è l'Underfitting?**
Si verifica quando il modello è troppo semplice per catturare la struttura logica dei dati. Si manifesta con performance scadenti sia sul Training Set che sul Test Set.
- **Come verifichiamo la presenza di overfitting o underfitting?**
Addestriamo il modello sul Training Set (80% del dataset, stratificato) e calcoliamo le metriche di Accuracy, Recall e AUC sia sul Training Set stesso che sul Test Set (20% rimanente, tenuto separato). Un divario marcato tra le metriche di train e test indica overfitting; valori bassi in entrambi indicano underfitting.

4.2 Caratterizzazione dei due Cluster ($k = 2$)

A seguito del clustering non supervisionato k -Means con $k = 2$ su tutti i dati preelaborati, abbiamo analizzato le caratteristiche dei due segmenti di clienti incrociandoli con le variabili non riscalate del dataset:

1. **Cluster 0 (Dimensioni: 1526 clienti - "Low-Spenders Fedeli"):**

- Tenure media: 30.55 mesi.
- Spesa mensile media (*MonthlyCharges*): 21.08\$.
- Costo totale medio (*TotalCharges*): 662.60\$.
- Tasso di Churn reale: **7.40%** (estremamente basso).
- Contratti: Prevalenza di contratti a lungo termine (Biennali: 41.8%, Annuali: 23.9%, Mensili: 34.3%).
- *Profilo*: Clientela stabile che utilizza servizi di base (es. solo linea voce, senza internet veloce), caratterizzata da elevata fedeltà e contratti pluriennali.

2. Cluster 1 (Dimensioni: 5517 clienti - “High-Spenders Volatili”):

- Tenure media: 32.88 mesi.
- Spesa mensile media (*MonthlyCharges*): 76.84\$.
- Costo totale medio (*TotalCharges*): 2727.03\$.
- Tasso di Churn reale: **31.83%** (molto alto).
- Contratti: Forte prevalenza di contratti flessibili mensili (Mese-a-mese: 60.7%, Annuali: 20.1%, Biennali: 19.2%).
- *Profilo*: Clientela ad alto consumo tecnologico (servizi internet a banda larga, streaming, pacchetti dati), ma con un’altissima volatilità e legata a contratti senza vincoli temporali.

5 Risultati

5.1 Risultati del Clustering k-Means

La Tabella 1 mostra i valori di Inertia e Silhouette Score per le diverse configurazioni di cluster testate.

Tabella 1: Metriche di valutazione del clustering k-Means.

Numero di Cluster (k)	Inertia	Silhouette Score
2	41817.82	0.2946
3	30802.81	0.2861
4	28228.08	0.2279

Il valore di silhouette score massimo si registra per $k = 2$ (0.2946). Per $k = 3$ si ha 0.2861, mentre per $k = 4$ scende a 0.2279. Abbiamo pertanto selezionato $k = 2$ come partizione ottimale.

La Figura 3 mostra l’andamento dell’Inertia e del Silhouette Score al variare del numero di cluster k .

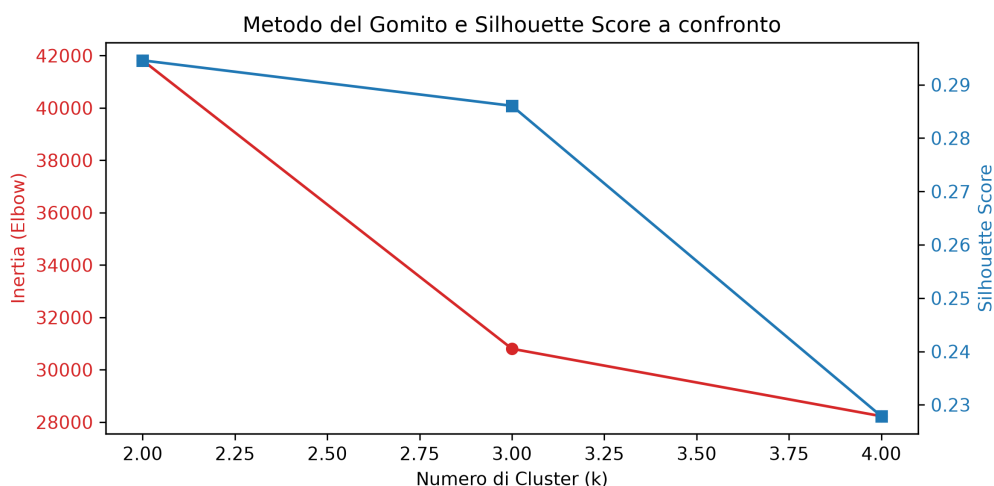


Figura 3: Valutazione del clustering k-Means tramite metodo del gomito (Inertia) e Silhouette Score.

5.2 Risultati della Classificazione (Train vs. Test)

La Tabella 2 riassume le performance di tutti i modelli sia sul Training Set che sul Test Set, nei due scenari.

Tabella 2: Confronto delle metriche di classificazione tra Train e Test.

Modello	Feature Scenario	Accuracy		Recall		AUC	
		Train	Test	Train	Test	Train	Test
Logistic Regression	Senza Cluster	0.8053	0.8062	0.5485	0.5588	0.8492	0.8422
Logistic Regression	Con Cluster	0.8051	0.8055	0.5485	0.5588	0.8492	0.8423
Decision Tree	Senza Cluster	0.8023	0.7942	0.5612	0.5401	0.8476	0.8267
Decision Tree	Con Cluster	0.8023	0.7942	0.5612	0.5401	0.8476	0.8267
Multi-Layer Perceptron (MLP)	Senza Cluster	0.9143	0.7495	0.7813	0.4278	0.9656	0.7844
Multi-Layer Perceptron (MLP)	Con Cluster	0.9104	0.7360	0.9097	0.5829	0.9722	0.7846

5.3 Analisi Critica dell'Overfitting e dell'Impatto del Clustering

L'ispezione della Tabella 2 evidenzia risultati estremamente interessanti ai fini dell'analisi:

1. Logistic Regression e Decision Tree (Modelli stabili):

Entrambi i modelli mostrano un comportamento ideale dal punto di vista della generalizzazione. La differenza tra le metriche di Training e Testing è minima (es. per la Regressione Logistica senza cluster, l'accuratezza passa da 0.8053 a 0.8062, e l'AUC da 0.8492 a 0.8422). **Non vi è alcun segno di overfitting.** Il Decision Tree beneficia dell'impostazione di `max_depth=5` che ha bloccato la crescita incontrollata dei rami. L'aggiunta della feature del cluster non altera le performance di questi modelli, in quanto la barriera di decisione del cluster $k = 2$ è già intrinsecamente ricavabile dalle feature originali.

2. Multi-Layer Perceptron - MLP (Forte Overfitting):

La rete MLP mostra un **evidente e forte overfitting**. Nel training set senza cluster, la rete raggiunge un'accuratezza del 91.43%, una Recall del 78.13% e un AUC del 0.9656. Tuttavia, quando valutata sul Test Set, l'accuratezza crolla al 74.95%, la Recall si dimezza al 42.78% e l'AUC scende a 0.7844. Le reti neurali MLP sono approssimatori molto complessi ed altamente parametrizzati; su dataset di medie dimensioni come questo (≈ 5600 campioni di training), tendono a memorizzare il rumore del dataset di training anziché astrarre pattern generali.

3. Impatto del Clustering sull'MLP:

L'aggiunta della feature di cluster nello Scenario B non risolve l'overfitting dell'MLP (la Train Accuracy rimane al 91.04%), ma agisce come una forte guida per l'attivazione dei neuroni. Sul Test Set, la **Recall dell'MLP con cluster compie un balzo notevole passando da 0.4278 a 0.5829 (+15.51% assoluto)**. Questo indica che l'informazione di partizione non supervisionata aiuta a compensare lo sbilanciamento delle classi, spingendo la rete neurale a individuare molti più clienti a rischio effettivo di abbandono, a costo di un lieve incremento di falsi positivi (lieve flessione dell'accuratezza a 0.7360).

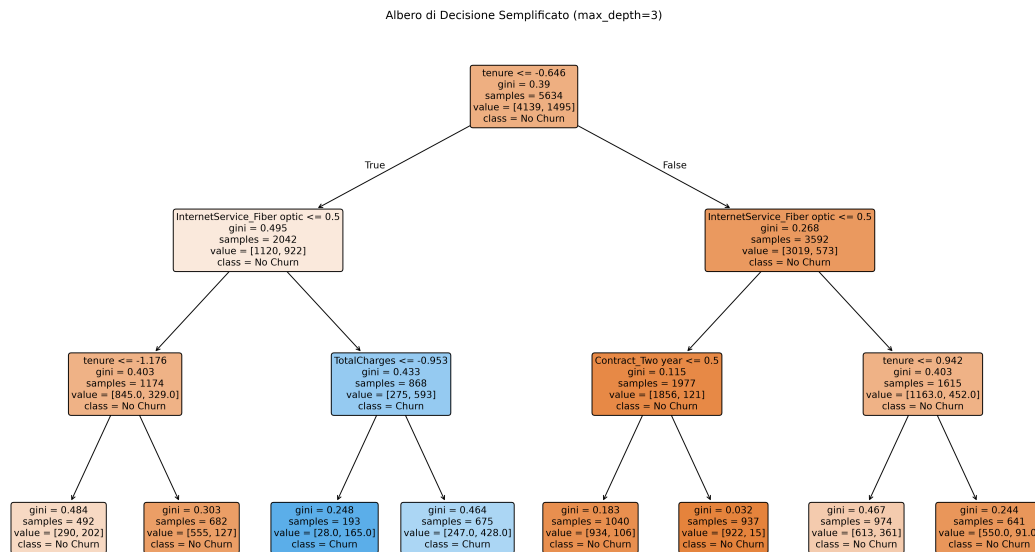


Figura 4: Albero di Decisione semplificato (max_depth=3).

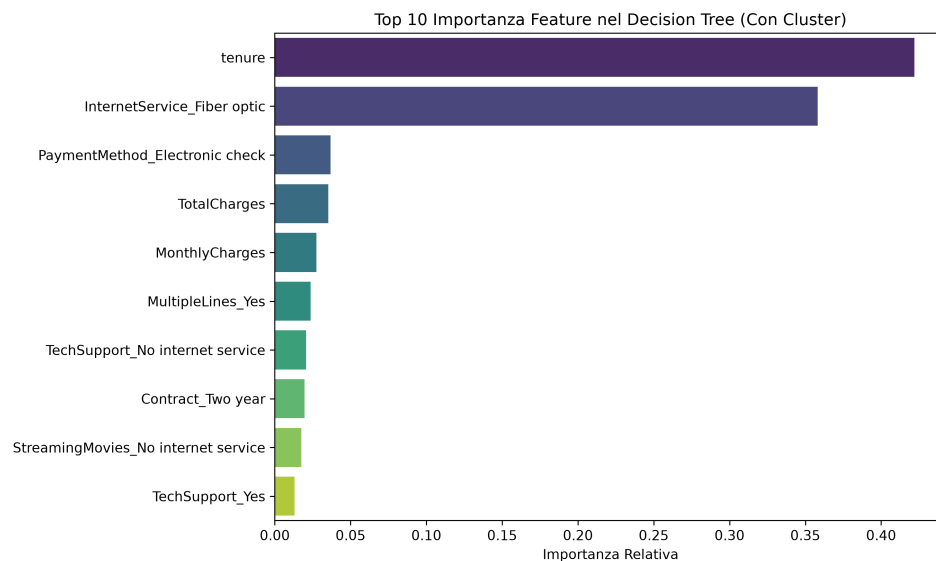


Figura 5: Top 10 importanza delle feature per il Decision Tree nello Scenario B (con feature del cluster).

6 Discussione critica

6.1 Punti di Forza della Soluzione

L'integrazione di clustering e classificazione ha consentito di caratterizzare i segmenti di clientela in base ai loro profili commerciali reali, offrendo modelli predittivi stabili (baseline e albero) e fornendo un incremento prestazionale importante per la Recall dell'MLP attraverso la feature di clustering.

6.2 Limiti Sperimentali e Fonti di Errore

I cluster identificati presentano confini deboli nello spazio originario (Silhouette Score di 0.2946), a indicare comportamenti di clientela sfumati. Inoltre, la rete MLP risente fortemente di overfitting su questo dataset, suggerendo la necessità di inserire tecniche di regolarizzazione (L2 aumentata, dropout) o Early Stopping.

6.3 Interpretazione e Confronto: Decision Tree vs. MLP

Il Decision Tree mantiene una totale trasparenza decisionale (struttura a IF-THEN basata principalmente sul tipo di contratto mensile e sulla tenure standardizzata), facilmente spiegabile. L'MLP, pur fornendo un'ottima Recall nello Scenario B, agisce come una scatola nera (black box) dove le relazioni tra i pesi dei nodi rimangono non interpretabili per un operatore di business.

6.4 Miglioramenti Futuri

Proponiamo l'applicazione di algoritmi di bilanciamento delle classi (SMOTE o pesatura della perdita tramite class weight) per innalzare la Recall senza degradare l'accuratezza, e l'utilizzo di modelli ensemble come il Random Forest.

6.5 Uso di strumenti di Intelligenza Artificiale Generativa

In conformità con le linee guida per la stesura dei progetti di laboratorio, si dichiara che per lo sviluppo di questo elaborato sono stati impiegati sistemi di Intelligenza Artificiale Generativa (nello specifico, Large Language Models). Tali strumenti sono stati utilizzati unicamente come supporto per la formattazione tipografica in $\text{\LaTeX}$ e per la revisione strutturale dei testi.

7 Conclusioni

Il progetto ha dimostrato l'utilità dell'integrazione tra apprendimento non supervisionato e supervisionato. Il clustering ha identificato segmenti commerciali ben caratterizzati (low-spenders stabili vs. high-spenders volatili). La feature derivata ha consentito di mitigare l'effetto dello sbilanciamento sul Multi-Layer Perceptron (MLP), incrementando del +15.5% il richiamo dei clienti a rischio di abbandono sul Test Set, a fronte di un evidente overfitting del modello neurale che evidenzia la necessità di regolarizzazioni future.